

DOCUMENTATION TECHNIQUE

Vite & Gourmand

1. Réflexions initiales technologiques :

Dans le cadre du projet *Vite & Gourmand*, une application web de gestion de commande de menus traiteur, plusieurs choix technologiques ont été étudiés avant le démarrage du développement :

Analyse des besoins :

L'application doit permettre :

- La visualisation des prestations proposées
- La création de compte utilisateurs ainsi que la connexion et déconnexion
- La gestion des utilisateurs (clients, employés, administrateurs)
- La gestion des menus et plats
- La gestion des commandes avec suivi des états
- La gestion des avis clients
- L'affichage de statistiques

Le projet nécessite donc :

- Un environnement web adapté et responsive
- Une architecture structurée
- Une base de données relationnelle
- Une base de données non relationnelle pour les statistiques
- Une séparation claire entre logique métier et affichage

Choix des technologies :

Front-end : **HTML**, **CSS** (Bootstrap) et **JavaScript**

- Affichage à partir de navigateur web (ordinateur et mobile)
- Mise en forme des pages facilitée et moderne
- Affichage dynamique de contenus

Back-end : **PHP** avec utilisation de PDO et Framework **Symfony**

- Application du modèle MVC (Modèle Vue Contrôleur)
- Adapté aux projets web professionnels
- Sécurise l'authentification et les formulaires
- Assure maintenabilité et évolutivité

Bases de données : **MariaDB** (MySQL) et **MongoDB** (NoSQL)

- MariaDB pour son côté relationnel qui gère les relations prédéfinies entre les données, organisées en tables, colonnes et lignes.

- **MongoDB** pour sa nature non relationnelle orienté documents qui stocke les données (paires de champs et valeurs) dans des documents de manière comparable aux objets JSON (JavaScript Object Notation).

Déploiement : **AlwaysData**

- Permet un hébergement simple, fiable et compatible avec le stack PHP/Symfony/MariaDB.
- Possibilité d'installer des extensions non disponible par défaut comme « mongodb ».

2. Configuration de l'environnement de travail :

Poste de travail :

- Ordinateur sous système d'exploitation **Windows**
- Navigateur web principal : **Google Chrome**
- Suite logiciel **XAMPP** (Apache, MySQL, FileZilla)
- Environnement de développement : **PHPStorm 2024.3**
- Gestion de versions : **Git** (Github)
- Virtualisation de l'environnement : **Docker**.

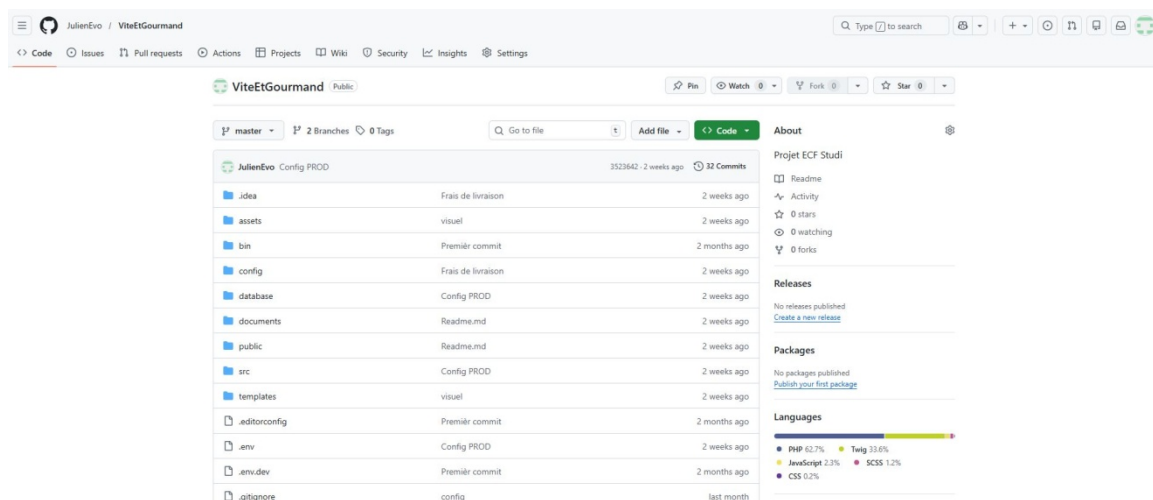


Figure 1 - Github (branche master)

Environnement local :

- **PHP 8.2**
- **Symfony 7.4**
- **Composer** pour la gestion des dépendances
- **Apache 2.4**
- Paramétrage des variables d'environnement dans le fichier **.env**

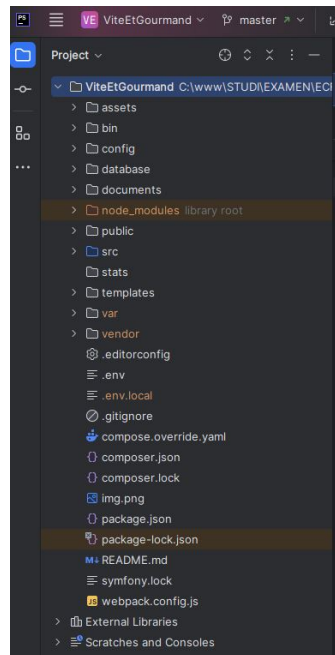


Figure 2 - Structure Symfony (PhpStorm)

Bases de données :

- **MariaDB 10.4** pour la base de données relationnelle
- **PhpMyAdmin** comme outil d'administration de MariaDB
- Création de la base de données via import d'un script SQL
- **MongoDB 2.1** pour la base de données non relationnelle
- **MongoDB Compass** comme outil d'administration de MoongODB

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ allergene	Parcourir Structure Rechercher Insérer Vider Supprimer	8	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ avis	Parcourir Structure Rechercher Insérer Vider Supprimer	8	InnoDB	utf8mb4_unicode_ci	48,0 kio	-
☐ commande	Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_unicode_ci	64,0 kio	-
☐ commande_etat	Parcourir Structure Rechercher Insérer Vider Supprimer	9	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ horaire	Parcourir Structure Rechercher Insérer Vider Supprimer	7	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ menu	Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ menu_plat	Parcourir Structure Rechercher Insérer Vider Supprimer	15	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ menu_regime	Parcourir Structure Rechercher Insérer Vider Supprimer	7	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ regime	Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ plat	Parcourir Structure Rechercher Insérer Vider Supprimer	26	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ plat_allergene	Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ plat_type	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ regime	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ societe	Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ societe_horaire	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ theme	Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ utilisateur	Parcourir Structure Rechercher Insérer Vider Supprimer	6	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
17 tables Somme		138	InnoDB	utf8mb4_unicode_ci	448,0 kio	0 o

Figure 3 – PhpMyAdmin

3. Modèle conceptuel de données (MCD) :

Le MCD (Modèle Conceptuel de Données) représente l'ensemble des entités ainsi que leurs relations et permet de visualiser clairement :

- Les entités principales (ex : utilisateur, commande, menu, plat), représentées par des rectangles avec leurs attributs

- Les associations entre elles (ex : un utilisateur peut passer plusieurs commandes), identifiées par des ovales
- Les cardinalités pour chaque relation (1:1, 0:N, 1:N)

Cette représentation a été réalisée avec l'application **Looping**, en respectant la méthode *Merise*.

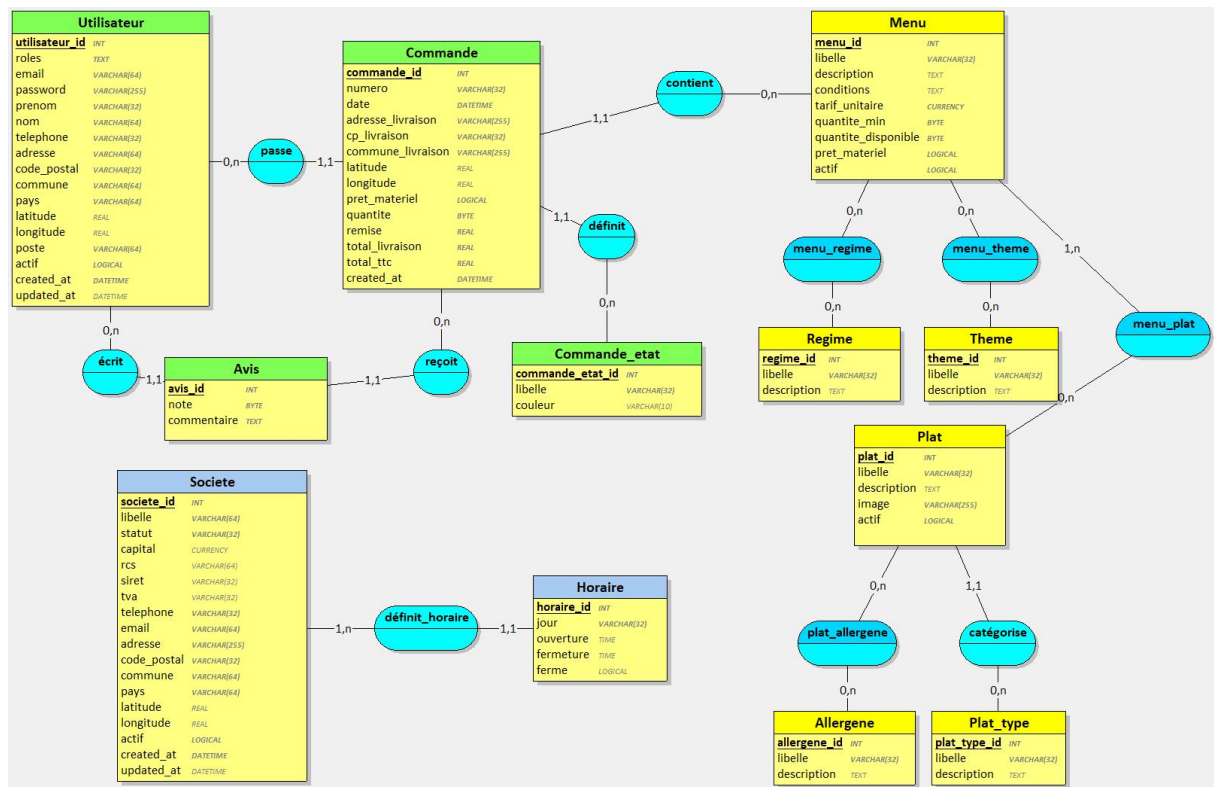


Figure 4 - MCD (Export Looping)

4. Diagramme de cas d'utilisation :

Le diagramme de cas d'utilisation permet de représenter les interactions entre les différents acteurs (visiteurs, utilisateurs, employé et administrateurs) et les fonctionnalités proposées par l'application qui donne une vision globale sur les droits de chaque acteur.

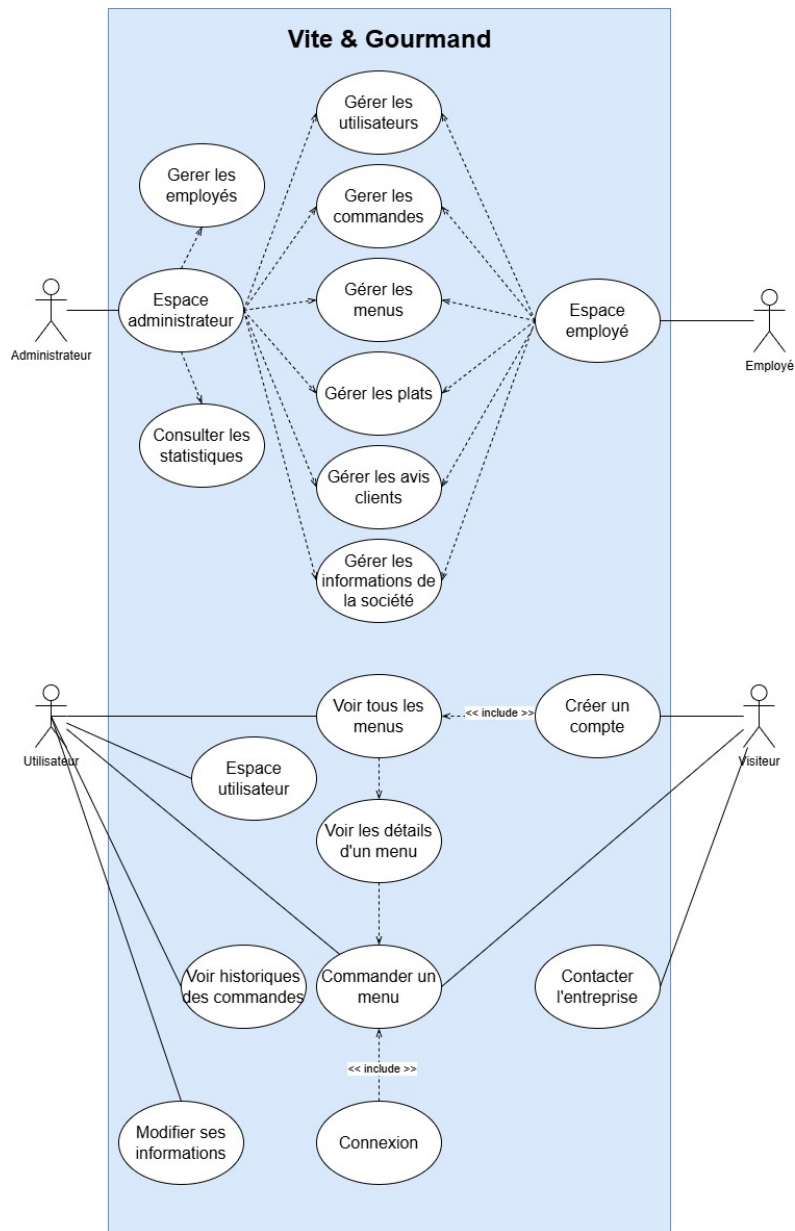


Figure 5 - Diagramme de cas d'utilisation

5. Diagramme de séquence – Validation d'une commande :

Le diagramme de séquence ci-dessous illustre le déroulement technique lorsqu'un client valide une commande dans l'application.

Il met en évidence la chronologie des interactions entre les différents composants du système, et garantit une compréhension précise du fonctionnement interne.

Le scénario fait intervenir :

- **Le client** : Utilisateur connecté avec ses identifiants
- **Le navigateur** : Interface entre l'utilisateur et l'application
- **CommandeController** : Contrôleur du modèle MVC, qui gère la logique métier
- **CommandeRepository** : Permet les interactions avec la base de données relationnelle
- **Bases de données** : Enregistrement persistant des données

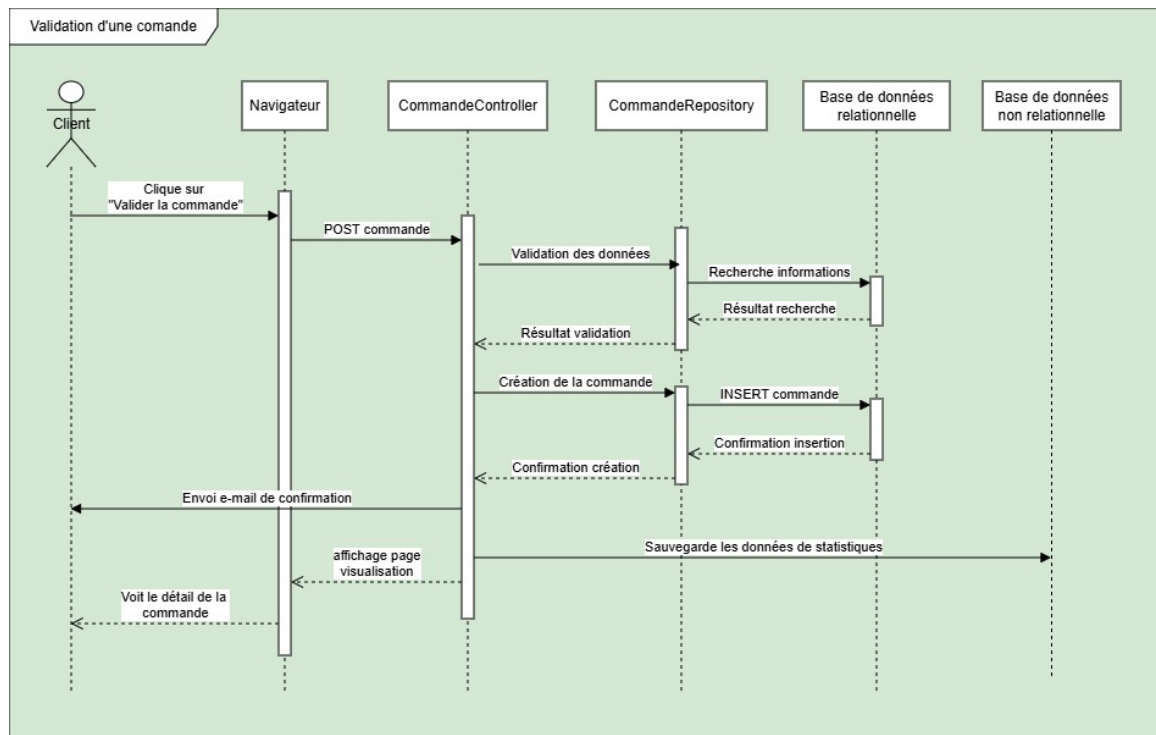


Figure 6 - Diagramme de séquence (Validation d'une commande)

Déroulement du processus :

1. Le client clique sur le bouton « valider la commande »
2. Le navigateur envoie une requête HTTP de type **POST** vers la route `/commande_validation`
3. Le *CommandeController* réceptionne la requête et effectue la validation des données en appelant le *CommandeRepository* afin d'interroger la base de données
4. Le repository renvoi les résultats des requêtes au contrôleur pour validation
5. *CommandeController* crée la commande et la transmet au repository pour l'enregistrement
6. Le service exécute une requête d'insertion (INSERT) en base de données
7. La base de données confirme l'enregistrement
8. Le contrôleur envoi un e-mail de confirmation au client
9. Le contrôleur enregistre des informations de statistiques en base de données NoSQL
10. Le contrôleur génère une réponse HTTP 200 avec la vue de visualisation de la commande
11. Le navigateur affiche la page de visualisation de la commande au client

6. Déploiement de l'application en production :

Le projet a été développé en environnement local avant d'être déployé en production. Cette démarche permet :

- De développer et tester sans impacter le site en production
- De corriger les erreurs avant mise en production
- De garantir la stabilité de l'application

Hébergeur :

Le déploiement a été réalisé sur un hébergement mutualisé AlwaysData (www.alwaysdata.com/fr) pour :

- Sa compatibilité avec PHP 8 et MariaDB
- La gestion simple des variables d'environnement
- La configuration facile des différents services (base de données, accès distant, installation d'extensions)

Étapes du déploiement :

- Création du compte *vite-et-gourmand* et ajout d'un nouveau site à partir de l'interface d'administration de l'hébergeur
- Configuration de l'adresse : *vite-et-gourmand.alwaysdata.net*
- Configuration du répertoire racine et de la version de PHP
- Création de la base de données et import du script SQL pour ajouter la structure et les données initiales
- Installation manuelle de l'extention MongoDB via **PECL** (PHP Extension Community Library) avec la commande *ad_install_pecl mongodb*. Ce qui va créer un fichier *mongodb.so* qui devra être renseigné comme extension dans le fichier *php.ini*.
- Ajout des variables d'environnement (bdd, configuration mail, API, production)
- Transfère des fichiers du projet via FTP (FileZilla)
- Installation des dépendances via SSH
- Test fonctionnels post-déploiement

Tests après mise en production :

Après le déploiement, plusieurs tests ont été effectués :

- Création de compte utilisateur
- Connexion / Déconnexion
- Visualisation des menus
- Création de commande
- Accès aux espaces utilisateur et administrateur
- Vérification de l'enregistrement en base de données

Ces vérifications permettent de garantir la conformité entre l'environnement local et production.

Les instructions détaillées d'installation sont disponible dans le fichier README.md du projet. La présente section synthétise la démarche technique adoptée.